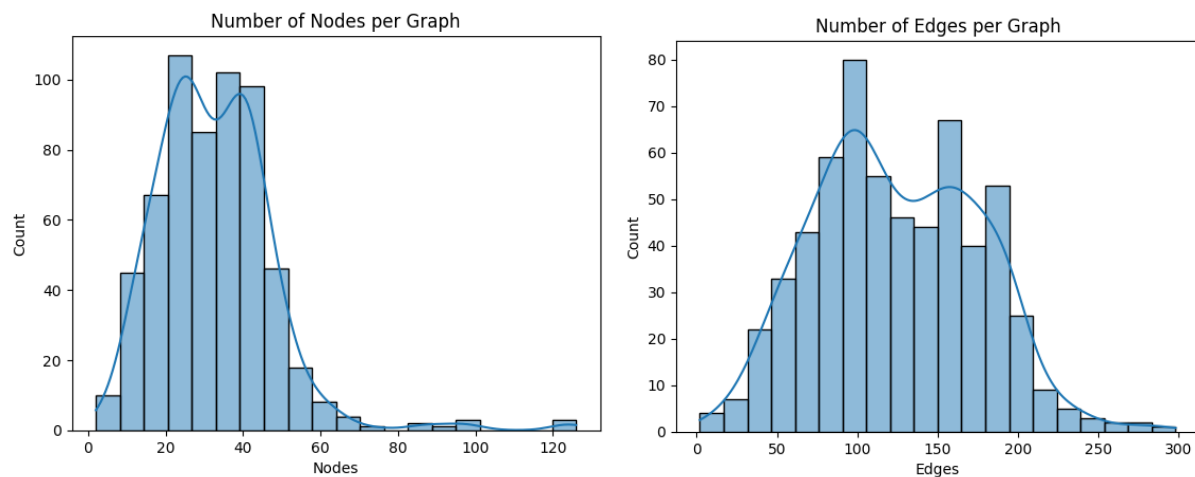


Homework 4: Graph Classification

Step 1. Data

1) This dataset has 600 graphs in total, and each of the graph belongs to one of the 6 classes (labels). The total number of nodes across all graphs is 19580, and the total number of edges is 74564. Each graph has different number of nodes and edges

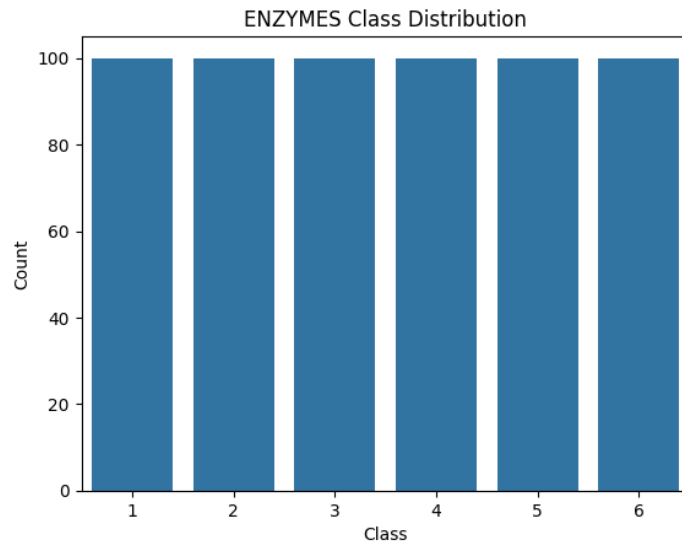


2) This dataset addresses the graph-level classification problem. The goal is predict which enzyme class a graph belongs to (6 classes)

3) Nodes represent atoms or components of the proteins, edges represent the relationship between adjacent atoms. The node features in this dataset has 18 dimensions and are continuous values. Additionally, there're also 3 categorical node labels (1-3)

4) There are no missing values in the node features

5) Each graph has a label (1-6)



6) Since the dataset is quite small, I will use 80% for training, 10% for validation and testing. I use stratified splitting to keep class balance.

7) For preprocessing, first I will convert all the label to 0-based since most of them are 1-based. Then the node labels will be one hot encoded and concatenated with the node features to enhance them further.

To construct the input graphs, the adjacency matrix will be converted into standard undirected graph format, and then concat with the label to produce a features – label pair for every protein graph

Step 2. Model

For this task, I implemented and trained GCN from scratch with PyTorch, experimenting with different number of layers.

For the architecture setup, each layer includes a graph convolution, batch normalization (this actually boosts the performance in my case), an activation, and a dropout layer. After several graph convolution layers, the node embeddings are pooled into a single graph-level representation. There are several options for the pooling method, such as global mean, global add, global max pool, each capturing different information. In my implementation, I concatenate the outputs from all 3 pooling methods to allow the model capture different aspects of the graph's structure. Finally, a small MLP classification head maps the pooled graph embedding to the 6 classes.

Model	# Parameters	F1	Accuracy
GCN-1 layer	204.6K	0.585	0.600
GCN-2 layer	270.9K	0.833	0.833
GCN-3 layer	337.2K	0.707	0.717

Step 3: Objective

For loss function, I use Cross Entropy Loss since it's the standard loss for multi-class classification.

Step 4: Optimization

For optimization, I select AdamW because it is an improved version of the traditional Adam optimizer. AdamW is said to provide better regularization and generalization compared to Adam.

Step 5: Model selection

Additionally, I employ ReduceLROnPlateau learning rate scheduler, which is configured to monitor the validation F1 score. The idea is when val F1 stops improving, the learning rate will be reduced by a factor of 0.5. This allows the model to take larger steps in the early stages and smaller steps later, which may help improve convergence.

The initial learning rate is set to $5e-4$, with weight decay of $1e-4$. This relatively small learning rate is chosen because it helps the optimization process to descend the loss landscape more gradually and stably. In the earlier experiments with larger learning rates, I observe that the training and validation losses fluctuated heavily, which shows unstable updates and difficulty converging.

Therefore, my final hyperparameters for training all models are:

- Hidden dimension = 256
- Dropout = 0.3
- Batch size = 32
- Epochs = 300
- Early stop with patience = 30

With this setup, GCN with 2 layer has the best performance

Step 6: Model performance

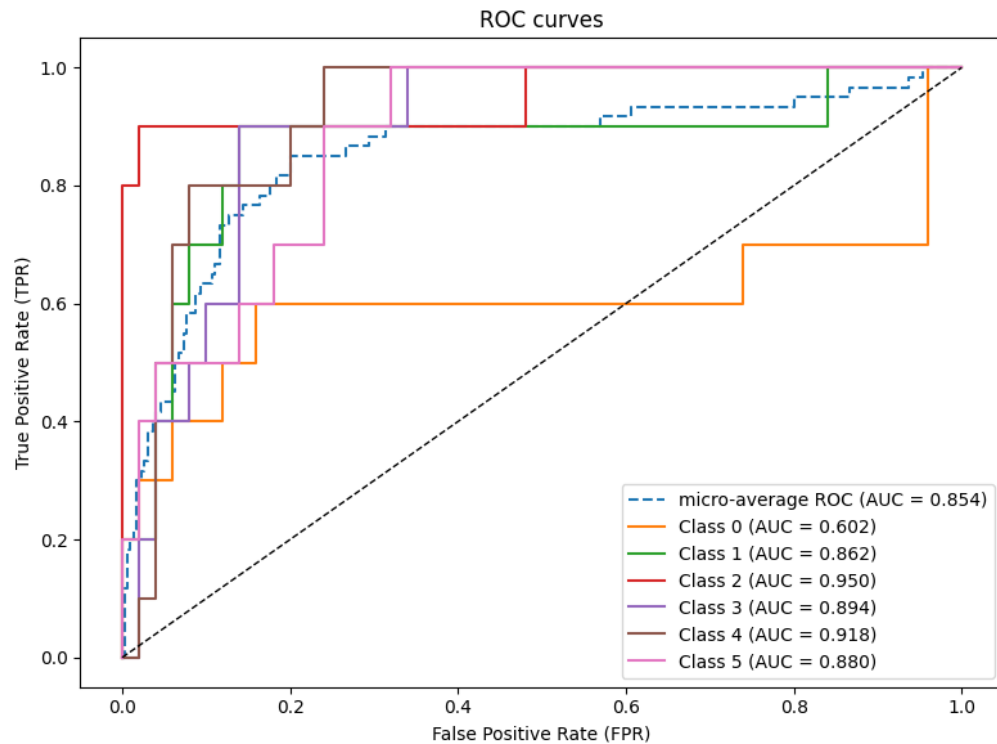


Figure 1. AUC curve of GCN-1 layer (Worst)

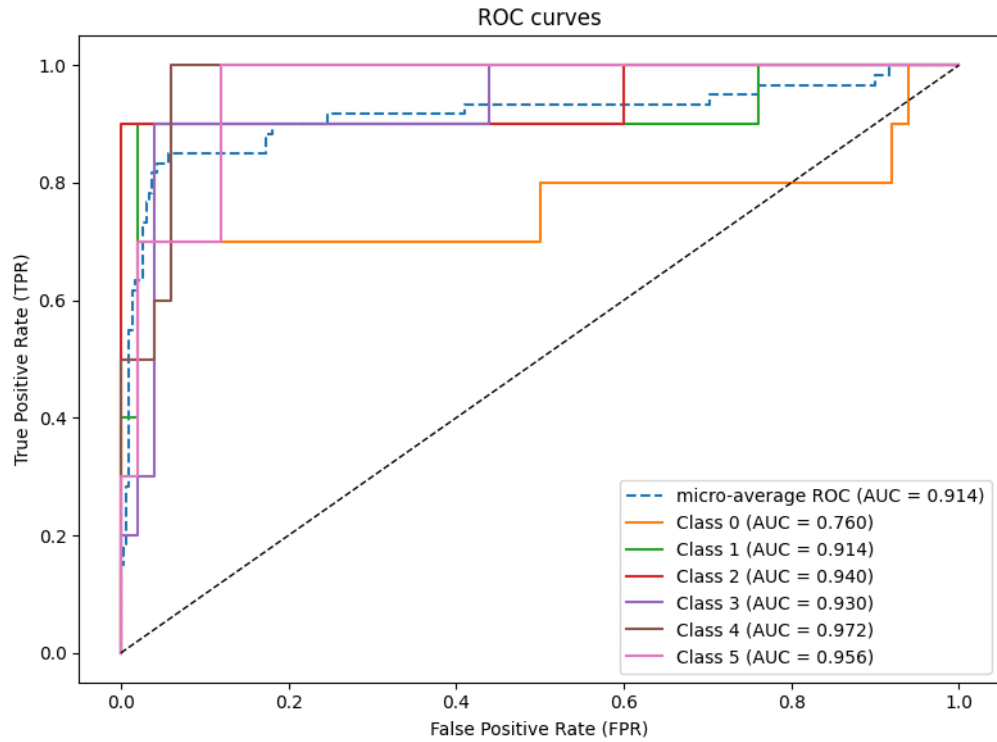


Figure 2. AUC curve of GCN-2 layer (Best)

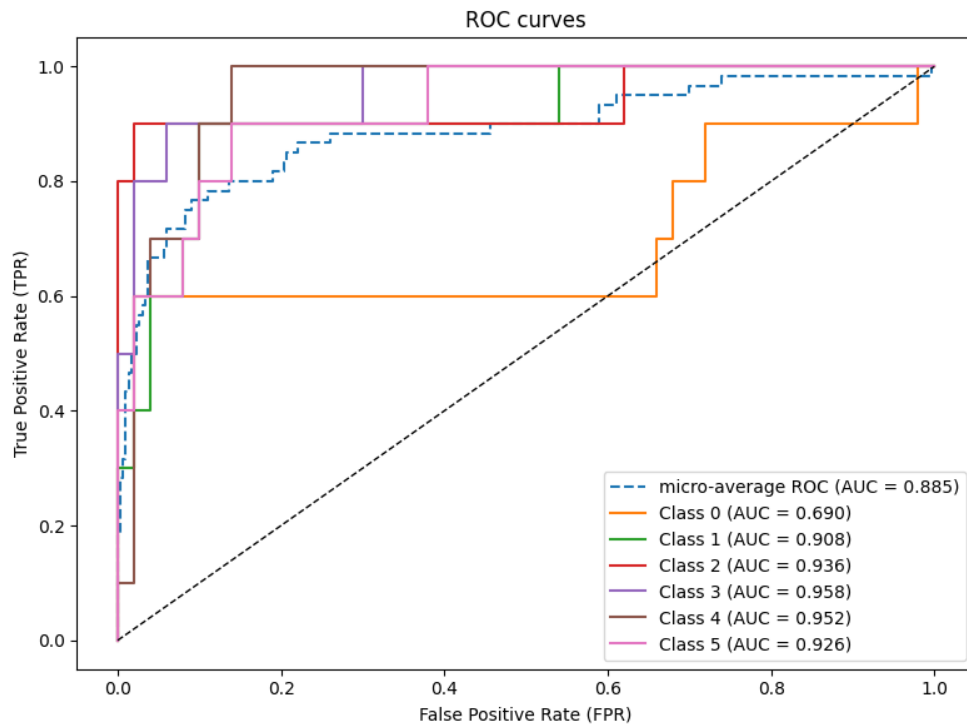


Figure 3. AUC curve of GCN-3 layer